

Week 7 - 8. Restaurant Chooser, Part 2

In the previous lab, we began dissecting the Restaurant Chooser application, to see what makes it tick. You looked at the basic structure of the code at a high level, the main entry point code, and the Restaurants screens (which, by extension, effectively showed you the People screen, because the code is practically identical to that of the Restaurants screen). That leaves a sizable chunk of the code to look at, the main code, really, in the Decision screen. This screen is, in fact, a group of sub-screens: **It's Decision Time**, **Who's Going**, **Pre-Filters**, **Choice**, and **Post-Choice**. These are the screens, in sequence, seen as the user uses the app to decide on a restaurant. (There's also the **Veto screen** that may or may not come after the Choice screen.) That's what we'll be looking at in this lab. However, before we start tearing the code apart, let's talk about **layout and flexbox**.

Let's Talk Layout and Flexbox

Flexbox is a layout algorithm that was introduced to CSS. It was designed with modern practices in mind, meaning such things as responsive design and layouts “flexing” (hence the name), based on the sizes of the components being laid out, even when those sizes are unknown or dynamic.

When dealing with flexbox, you are concerned with some parent container element, a component in React Native, and its immediate children, and how they are arranged in one direction or another. Components that use flexbox to lay their children out can then have one or more of those children themselves use flexbox to lay their children out, and so on, as far as you have to go. In this way, by nesting components that use flexbox, you can achieve pretty much any layout you can envision.

All that is required to use flexbox for layout is to specify some style attributes on a component. As mentioned before, the most commonly used attributes are `flex`, `justifyContent`, `alignItems`, and `flexDirection`. There are a few others that are important, but somewhat less so, including `alignSelf`, `flexWrap`, `alignContent`, `position`, and `zIndex`. By and large, if you find a reference about flexbox on the Web, it will apply to React Native as well, though there are a few differences you should be aware of.

First, the defaults are a little different in React Native. With flexbox on the Web, the default for `flexDirection` is `row`, but in React Native, it's `column`. Second, the `flex` parameter only supports a single number. Okay, that's all well and good, but what do these attributes do? Let's look at each now. The first attribute is `flexDirection` (valid values: `row`, `column`, `column-reverse`, and `row-reverse`, with `column` being the default). This defines the direction of the main, or primary, layout axis. In other words, this determines whether the children of the container will be laid out horizontally across the screen (when `flexDirection` is set to `row`) or vertically (when set to `column`). The altered default (altered from flexbox on the Web) makes sense in this context, because most components on a mobile device are laid out vertically down the screen. Next up is `flex`. This attribute tells flexbox how the available space along the primary axis will be apportioned to the children. This can get a little complicated, but let's start with the simple part: your main container will nearly always have a `flex` value of 1, which means it will take up the entire height of the screen (or the whole width, if you were laying out in row mode). Then, the children inside that container will divide the available space according to their `flex` values.

Let's say that you've got the following code:

```
import React from "react";
import { View } from "react-native";

export default class App extends React.Component {
  render() {
    return (
      <View style={{ flex : 1 }}>
        <View style={{ flex : 5, backgroundColor : "red" }} />
        <View style={{ flex : 2, backgroundColor : "green" }} />
        <View style={{ flex : 3, backgroundColor : "blue" }} />
      </View>
    );
  }
}
```

```

    </View>
  );
}
}

```

The first `View` has a `flex` value of 1, so it will fill the screen. It has three child `Views`, each with different `flex` values. How much space will they take up? Well, all three of them combined will fill the parent `View`, which means they will fill the screen, and that's because if you don't specify a height for the components, the `View` will stretch to fill space, based on its `flex` value, by default. But, how much of the screen's height will each `View` take? To know, you add up the `flex` values, ten here, and the `flex` value of each forms a fraction, using that value as the denominator. In other words, the first (red) child `View` takes up 5/10 of the screen, which means 1/2 (just reduce the fraction as you would mathematically). The second (green) child `View` takes up 2/10 of the screen, or 1/5. The third (blue) child `View` takes up 3/10 of the screen.

Let's say now that we change the `flex` values of the children to be 1, 2, and 3, respectively. Add those up, and we get 6, so now the first child takes up 1/5 of the screen, the second 2/5, and the third 3/5. If you did 4, 2, 4 instead, they now take up 2/5, 1/5, and 2/5, respectively. See how that works? You can also mix `flex` values with static values. For example, replace the `flex` attribute of the first child `View` with `height:100` and change the `flex` value of the other two children to 1, and you'll wind up with a red box 100 pixels in height, and the other two will evenly divide the remaining space between them. In this particular case, the actual `flex` values you use don't matter, only that they are the same matters.

Keeping this train going, we next come to `justifyContent` (valid values: `flex-start`, `flex-end`, `center`, `space-between`, and `space-around`, with `flex-start` being the default). This attribute tells flexbox how the children are distributed across the primary axis. A value of `flex-start` means the children will "bunch up" (meaning with no space between them) at the top (or left, depending on the primary axis) of their parent. A value of `flex-end` means the exact opposite; they'll bunch up on the right or bottom. A value of `center` means they'll bunch up in the middle of the container, with any available unused space above and below (or left and right) of the children. A value of `space-between` ensures that any unused space is distributed evenly between the children (with no unused space before and after the first and last child, correspondingly), and `space-around` ensures that available space is allocated between all children, including before and after the first and last child.

They say a picture is worth a thousand words, so have a look at Figure 1, to see the effect of these settings in action (when the default `flexDirection` of `column` is used, that is).

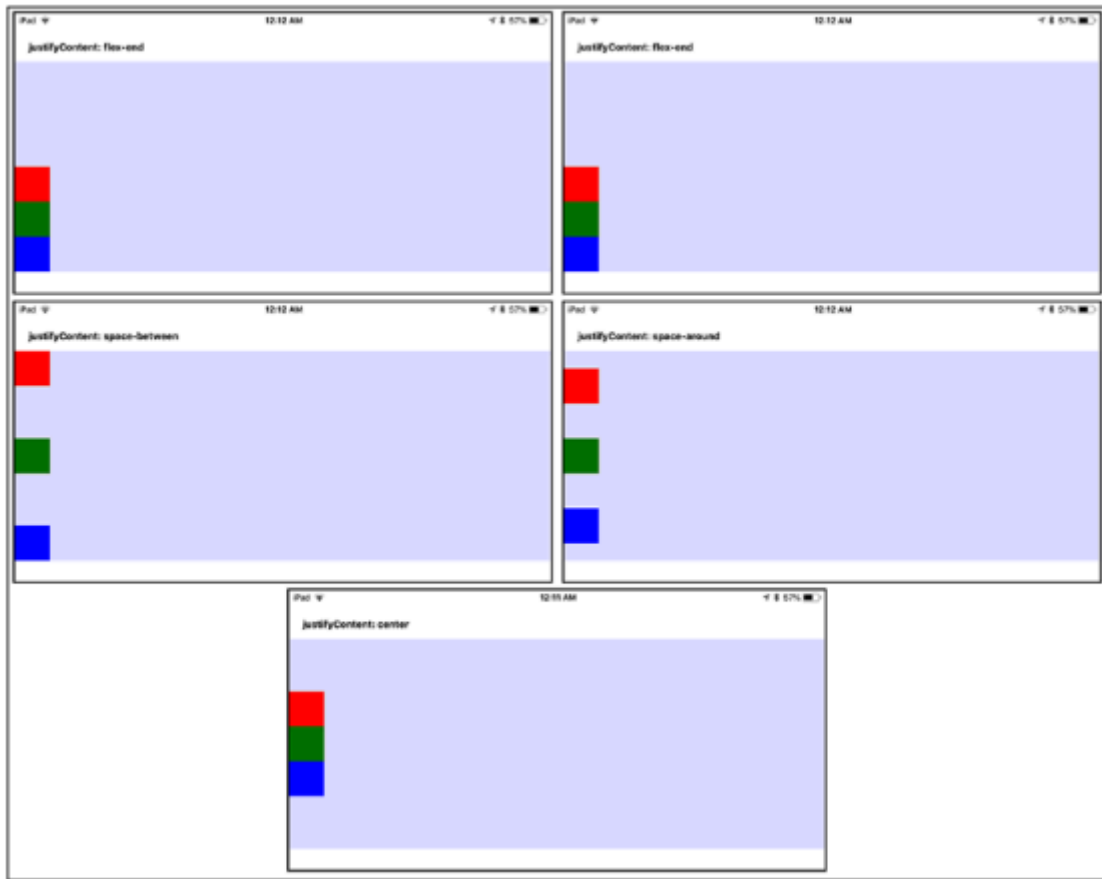


Figure 1. The various settings of `justifyContent`

Next up is the `alignItems` attribute (valid values: `flex-start`, `flex-end`, `center`, and `stretch`, with `stretch` being the default). This determines how the children align on the secondary, or “cross” axis. That means that, in the case of the default column layout, `alignItems` determines how the children align horizontally. Once again, let’s go to a picture, Figure 2, to make these settings clear.

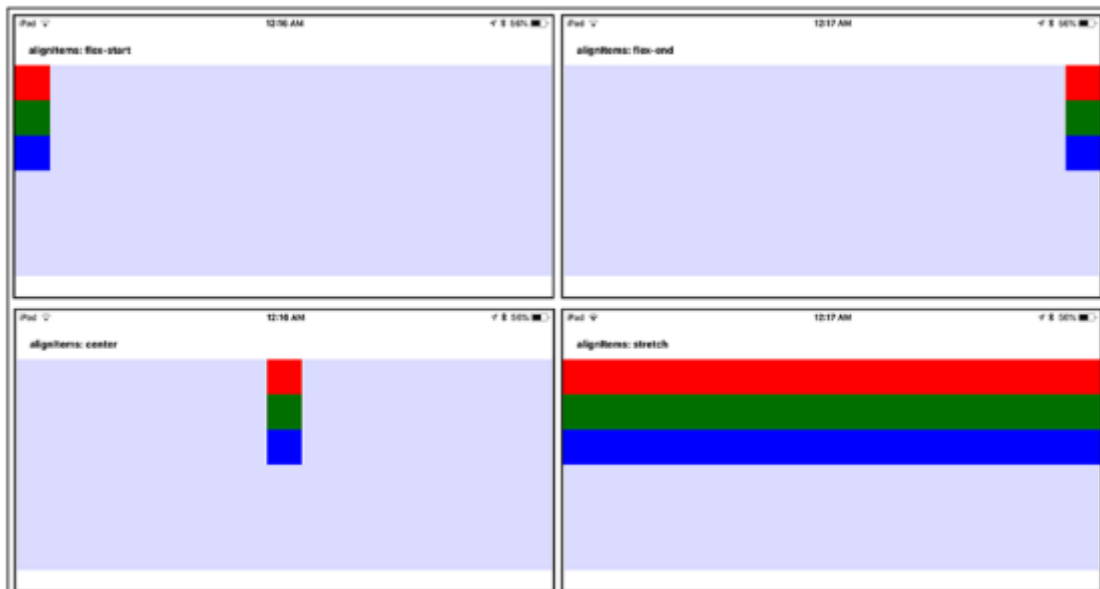


Figure 2. The various settings of `alignItems`

One important note is that for `flex-start`, `center`, and `flex-end` to work, your children must have specific widths. By contrast, they cannot have a fixed width, if you want to use `stretch`. The `alignSelf` attribute (valid values: `auto`, `flex-start`, `flex-end`, `center`, `stretch` or `baseline`, with `auto` being the default) allows a child to define its own `alignItems` value for it alone, overriding the parent’s `alignItems` value. This is a lesser used attribute, but

it's there if you need it. The `flexWrap` attribute (valid values: `wrap` or `nowrap`, with `nowrap` being the default) tells flexbox whether children are forced onto a single line, even if that means that they will flow off the screen (`nowrap`) or whether they can wrap onto a second line (`wrap`). Of course, whether your primary axis is horizontal or vertical determines whether you'll get a second row of children or a second column.

The `alignContent` attribute (valid values: `flex-start`, `center`, `flex-end`, `stretch`, `space-between`, `space-around`, with `flex-start` being the default) allows you to align the lines of children when `flexWrap` is set to `wrap` and when you wind up with multiple rows or columns. For example, if your layout flows to have two columns of children, setting `alignContent` to `center` ensures that both columns will be aligned to the center of the parent container. The final two attributes, `position` and `zIndex`, work just as they do on the Web: `position` can be either relative or absolute and controls whether items position themselves relative to the previous sibling (if any) or whether they are positioned based on absolute x/y coordinates relative to the parent's origin corner. The `zIndex` attribute allows you to place children on top of others (when `position` is set to absolute; otherwise, overlapping wouldn't occur when `position` is relative). By default, `position` is relative, just as on the Web. With this information, you should be able to achieve pretty much any layout in React Native that you require. However, note that there are more layout-related style attributes available that you might also make use of. You can have a look here (but note that the React Native docs refer to these as "Layout Props," although they're still just style attributes in the end): <https://facebook.github.io/react-native/docs/layout-props.html>.

Now, back to our application.

screens/decision

Think of the Decision folder as the restaurant choosing engine of your app. This is where all the magic happens! It's like a game flow with multiple screens:

Start → Who's going? → Any filters? → Pick random → Veto? → Final choice!

The Overall Flow

Imagine you're with friends trying to decide where to eat:

1. **DecisionTimeScreen**: The starting screen that checks if you have people and restaurants
2. **WhosGoingScreen**: Pick which friends are eating with you
3. **PreFiltersScreen**: Set preferences (cuisine type, price range, etc.)
4. **ChoiceScreen**: App picks a random restaurant, group can veto
5. **PostChoiceScreen**: Show the final chosen restaurant details

decisionScreenNavigation.js - The Stack Navigator

```
RestaurantChooser > screens > decision > JS decisionScreenNavigation.js > default
1  import React from "react";
2  import { createStackNavigator } from "@react-navigation/stack";
3  import DecisionTimeScreen from "../decisionScreen";
4  import WhosGoingScreen from "../whosGoingScreen";
5  import PreFiltersScreen from "../preFiltersScreen";
6  import ChoiceScreen from "../choiceScreen";
7  import PostChoiceScreen from "../postChoiceScreen";
8
9  const Stack = createStackNavigator();
10
11 const DecisionScreenNavigation = () => {
12   return (
13     <Stack.Navigator screenOptions={{ headerShown: false }}>
14       <Stack.Screen name="DecisionTimeScreen" component={DecisionTimeScreen} />
15       <Stack.Screen name="WhosGoingScreen" component={WhosGoingScreen} />
16       <Stack.Screen name="PreFiltersScreen" component={PreFiltersScreen} />
17       <Stack.Screen name="ChoiceScreen" component={ChoiceScreen} />
18       <Stack.Screen name="PostChoiceScreen" component={PostChoiceScreen} />
19     </Stack.Navigator>
20   );
21 };
22
23 export default DecisionScreenNavigation;
```

This is the roadmap for the decision process. Think of it like a stack of cards:

Screen	Purpose	Real-World Analogy
DecisionTimeScreen	Starting point, checks if you have data	The front door
WhosGoingScreen	Pick who's eating	Taking attendance
PreFiltersScreen	Set preferences (cuisine, price)	Discussing what everyone wants
ChoiceScreen	App picks, group vetoes	The voting round
PostChoiceScreen	Show final choice	Announcing the winner

The `headerShown: false` means no extra title bars - just clean screens stacking on top of each other.

decisionScreen.js - The Starting Point

```

1  import React from "react";
2  import AsyncStorage from "@react-native-async-storage/async-storage";
3  import {
4    Alert,
5    StyleSheet,
6    Text,
7    TouchableOpacity,
8    View,
9    Image,
10   } from "react-native";
11
12  const DecisionsScreen = ({ navigation }) => {
13    const handlePress = async () => {
14      try {
15        const people = await AsyncStorage.getItem("people");
16        const restaurants = await AsyncStorage.getItem("restaurants");
17
18        if (!people || JSON.parse(people).length === 0) {
19          Alert.alert(
20            "That ain't gonna work, chief",
21            "You haven't added any people. You should probably do that first, no?",
22            [{ text: "Ok" }],
23            { cancelable: false }
24          );
25        } else if (!restaurants || JSON.parse(restaurants).length === 0) {
26          Alert.alert(
27            "That ain't gonna work, chief",
28            "You haven't added any restaurants. You should probably do that first, no?",
29            [{ text: "Ok" }],
30            { cancelable: false }
31          );
32        } else {
33          navigation.navigate("WhosGoingScreen");
34        }
35      } catch (error) {
36        console.error("Failed to load data:", error);
37      }
38    };
39
40    return (
41      <View style={styles.decisionTimeScreenContainer}>
42        <TouchableOpacity
43          style={styles.decisionTimeScreenTouchable}
44          onPress={handlePress}>
45          <Image source={require("../assets/its-decision-time.android.png")} />
46          <Text style={{ padding: 20 }}>Click the food to get going</Text>
47        </TouchableOpacity>
48      </View>
49    );
50  };
51
52  const styles = StyleSheet.create({
53    decisionTimeScreenContainer: {
54      flex: 1,
55      alignItems: "center",
56      justifyContent: "center",
57    },
58    decisionTimeScreenTouchable: {
59      alignItems: "center",
60      justifyContent: "center",
61    },
62  });
63
64  export default DecisionsScreen;

```

This is the front door of the decision process. It's like a bouncer checking:

Step 1: User taps the food image

- App checks if you have ANY people saved
- App checks if you have ANY restaurants saved

Step 2: If missing data:

- Shows a friendly alert: "That ain't gonna work, chief"
- Tells you to add people or restaurants first

Step 3: If everything exists:

- Navigates to "WhosGoingScreen" to start the real process

The important check:

```
if (!people || JSON.parse(people).length === 0)
```

!people = No data at all

JSON.parse(people).length === 0 = Empty list

Either way, can't proceed!

whosGoingScreen.js - Pick Your Diners

```
1 import React, { useState, useEffect, useCallback } from "react";
2 import {
3   View,
4   Text,
5   FlatList,
6   TouchableOpacity,
7   Alert,
8   Platform,
9   BackHandler,
10 } from "react-native";
11 import AsyncStorage from "@react-native-async-storage/async-storage";
12 import Checkbox from "expo-checkbox";
13 import Constants from "expo-constants";
14 import { useNavigation, useFocusEffect } from "@react-navigation/native";
15 import CustomButton from "../../components/customButton";
16
17 const WhosGoingScreen = () => {
18   const [people, setPeople] = useState([]);
19   const [selected, setSelected] = useState([]);
20   const navigation = useNavigation();
21
22   useEffect(() => {
23     const loadPeople = async () => {
24       try {
25         const storedPeople = await AsyncStorage.getItem("people");
26         const parsedPeople = storedPeople ? JSON.parse(storedPeople) : [];
27         setPeople(parsedPeople);
28         setSelected(parsedPeople.map(() => false));
29       } catch (error) {
30         Alert.alert("Error", "Failed to load people data.");
31       }
32     };
33     loadPeople();
34   }, []);
35
36   // Handle hardware back button on Android
37   useFocusEffect(
38     useCallback(() => {
39       const onBackPress = () => {
40         Alert.alert("Confirm", "Do you want to go back?", [
41           { text: "Cancel", style: "cancel" },
42           {
43             text: "Yes",
44             onPress: () => navigation.goBack(),
45           },
46         ]);
47         return true; // prevent default behavior
48       };
49
50       const backHandlerSubscription = BackHandler.addEventListener(
51         "hardwareBackPress",
52         onBackPress
53       );
54       return () => backHandlerSubscription?.remove();
55     }, [navigation])
56   );
57
58   const toggleSelection = (index) => {
59     const updatedSelected = [...selected];
60     updatedSelected[index] = !updatedSelected[index];
61     setSelected(updatedSelected);
62   };
63
64   const handleText = () => {
65     const selectedParticipants = people
66       .map((person, index) => {
67         selected[index] ? { ...person, vetoed: "no" } : null
68       })
69       .filter(Boolean);
70
71     if (selectedParticipants.length === 0) {
72       Alert.alert("No participants", "Please select at least one person.");
73       return;
74     }
75
76     navigation.navigate("PreFiltersScreen", {
77       participants: selectedParticipants,
78     });
79   };
80
81   const renderItem = ({ item, index }) => (
82     <TouchableOpacity
83       style={styles.whosGoingItemTouchable}
84       onPress={() => toggleSelection(index)}
85     >
86       <Checkbox
87         value={selected[index]}
88         onValueChange={() => toggleSelection(index)}
89         style={styles.whosGoingCheckbox}
90       />
91       <Text style={styles.whosGoingName}>
92         {item.firstName} {item.lastName} {item.relationship}
93       </Text>
94     </TouchableOpacity>
95   );
96 }
```

```

95
96   return (
97     <View style={styles.listScreenContainer}>
98       <Text style={styles.whosGoingHeadline}>Who's Going?</Text>
99       <Flatlist
100         style={{ width: "94%" }}
101         data={people}
102         keyExtractor={(item) => item.key.toString()}
103         renderItem={renderItem}
104       />
105       <CustomButton text="Next" onPress={handleNext} />
106     </View>
107   );
108 };
109
110 const styles = {
111   listScreenContainer: {
112     flex: 1,
113     alignItems: "center",
114     justifyContent: "center",
115     padding: 20,
116     gap: 20,
117     ...Platform.select({
118       ios: {},
119       android: {},
120     }),
121   },
122   whosGoingHeadline: {
123     fontSize: 30,
124   },
125   whosGoingItemTouchable: {
126     flexDirection: "row",
127     marginTop: 10,
128     marginBottom: 10,
129     gap: 10,
130   },
131   whosGoingCheckbox: {},
132   whosGoingName: {
133     flex: 1,
134   },
135 };
136
137 export default WhosGoingScreen;

```

This is the attendance sheet for your dinner group:

State variables:

- people: List of all people from storage
- selected: Array of booleans (true = checked, false = not checked)

When screen loads (useEffect):

- Gets all people from AsyncStorage
- Creates a selected array with all false (nobody checked yet)
- Like starting with a blank attendance sheet

The checkbox logic:

run npm install expo-checkbox

```

const toggleSelection = (index) => {
  const updatedSelected = [...selected]; // Copy the array
  updatedSelected[index] = !updatedSelected[index]; // Flip this one
  setSelected(updatedSelected); // Save it
};

```

When user taps Next:

```

const selectedParticipants = people
  .map((person, index) =>
    selected[index] ? { ...person, vetoed: "no" } : null
  )
  .filter(Boolean);

```

This is clever! Let's break it down:

1. Look at each person and their checkbox status
2. If checked (selected[index] is true), keep that person AND add a vetoed: "no" property
3. If not checked, put null
4. .filter(Boolean) removes all the null values (keeps only checked people)

The result: An array of only the selected people, each with a fresh "vetoed: no" flag.

Android back button handling:

- When user presses hardware back button, asks "Are you sure?"
- Prevents accidentally leaving the screen

preFiltersScreen.js - Set Your Preferences

```
RestaurantChooser > screens > decision > .js preFiltersScreen.js > PreFiltersScreen
1 import React, { useState } from "react";
2 import { View, Text, ScrollView, Alert, Platform } from "react-native";
3 import AsyncStorage from "@react-native-async-storage/async-storage";
4 import { Picker } from "@react-native-picker/picker";
5 import Constants from "expo-constants";
6 import { useNavigation, useRoute } from "@react-navigation/native";
7 import CustomButton from "../../components/customButton";
8
```

```
8
9 const PreFiltersScreen = () => {
10   const [cuisine, setCuisine] = useState("");
11   const [price, setPrice] = useState("");
12   const [rating, setRating] = useState("");
13   const [delivery, setDelivery] = useState("");
14
15   const navigation = useNavigation();
16   const route = useRoute();
17   const { participants } = route.params;
18
19   const handleNext = async () => {
20     try {
21       const storedRestaurants = await AsyncStorage.getItem("restaurants");
22       const parsedRestaurants = storedRestaurants
23         ? JSON.parse(storedRestaurants)
24         : [];
25
26       const filteredRestaurants = parsedRestaurants.filter((restaurant) => {
27         const matchesCuisine = !cuisine || restaurant.cuisine === cuisine;
28         const matchesPrice =
29           !price || Number(restaurant.price) <= Number(price);
30         const matchesRating =
31           !rating || Number(restaurant.rating) >= Number(rating);
32         const matchesDelivery =
33           !delivery || restaurant.delivery === (delivery === "Yes");
34
35         return (
36           matchesCuisine && matchesPrice && matchesRating && matchesDelivery
37         );
38       });
39
40       if (filteredRestaurants.length === 0) {
41         Alert.alert(
42           "No Results",
43           "No restaurants match the selected criteria."
44         );
45       }
46       return;
47     }
48   }
49
```

```
47
48   navigation.navigate("ChoiceScreen", {
49     participantsList: participants,
50     filteredRestaurants,
51   });
52 } catch (error) {
53   Alert.alert("Error", "Failed to load or filter restaurants.");
54 }
55 }
56
```

```
return (
  <ScrollView style={styles.preFiltersContainer}>
    <View style={styles.preFiltersInnerContainer}>
      <View style={styles.preFiltersScreenFormContainer}>
        <View style={styles.preFiltersHeadlineContainer}>
          <Text style={styles.preFiltersHeadline}>Pre-Filters</Text>
        </View>
        <Text style={styles.fieldLabel}>Cuisine</Text>
        <View style={styles.pickerContainer}>
          <Picker
            selectedValue={cuisine}
            onChange={(value) => setCuisine(value)}
            style={styles.picker}
          >
            <Picker.Item label="Any" value="" />
            <Picker.Item label="Italian" value="Italian" />
            <Picker.Item label="Chinese" value="Chinese" />
            <Picker.Item label="Indian" value="Indian" />
            <Picker.Item label="Mexican" value="Mexican" />
            <Picker.Item label="Other" value="Other" />
          </Picker>
        </View>
      </View>
    </View>
  </ScrollView>
);
```

```

</View>

<Text style={styles.fieldLabel}>Max Price</Text>
<View style={styles.pickerContainer}>
  <Picker
    selectedValue={price}
    onValueChange={(value) => setPrice(value)}
    style={styles.picker}
  >
    <Picker.Item label="Any" value="" />
    <Picker.Item label="$" value="1" />
    <Picker.Item label="$ $" value="2" />
    <Picker.Item label="$ $ $" value="3" />
    <Picker.Item label="$ $ $ $" value="4" />
  </Picker>
</View>

<Text style={styles.fieldLabel}>Min Rating</Text>
<View style={styles.pickerContainer}>
  <Picker
    selectedValue={rating}
    onValueChange={(value) => setRating(value)}
    style={styles.picker}
  >
    <Picker.Item label="Any" value="" />
    <Picker.Item label="1 Star" value="1" />
    <Picker.Item label="2 Stars" value="2" />
    <Picker.Item label="3 Stars" value="3" />
    <Picker.Item label="4 Stars" value="4" />
    <Picker.Item label="5 Stars" value="5" />
  </Picker>
</View>

<Text style={styles.fieldLabel}>Delivery</Text>
<View style={styles.pickerContainer}>
  <Picker
    selectedValue={delivery}
    onValueChange={(value) => setDelivery(value)}
    style={styles.picker}
  >
    <Picker.Item label="Any" value="" />
    <Picker.Item label="Yes" value="Yes" />
    <Picker.Item label="No" value="No" />
  </Picker>
</View>

<CustomButton text="Next" onPress={handleNext} />

```

```

        </View>
    </View>
</ScrollView>
);
};

const styles = {
  preFiltersContainer: {},
  preFiltersInnerContainer: {
    flex: 1,
    alignItems: "center",
    paddingTop: 20,
    paddingBottom: 20,
    width: "100%",
  },
  preFiltersScreenFormContainer: {
    width: "96%",
  },
  preFiltersHeadlineContainer: {
    flex: 1,
    alignItems: "center",
    justifyContent: "center",
  },
  preFiltersHeadline: {
    fontSize: 30,
  },
  fieldLabel: {
    marginLeft: 10,
  },
  pickerContainer: {
    ...Platform.select({
      ios: {},
      android: {
        width: "96%",
        borderRadius: 8,
        borderColor: "#c0c0c0",
        borderWidth: 2,
        marginLeft: 10,
        marginBottom: 20,
        marginTop: 4,
      },
    }),
  },
  picker: {
    ...Platform.select({
      ios: {
        width: "96%",

```

```

borderRadius: 8,
borderColor: "#c0c0c0",
borderWidth: 2,
marginLeft: 10,
marginBottom: 20,
marginTop: 4,
},
android: {},
}),
},
};

export default PreFiltersScreen;

```

This is the filter screen - like telling Google Maps "only show me Italian restaurants under \$20":

State variables for each filter:

- cuisine: Type of food (Italian, Chinese, etc.)
- price: Maximum price level (1-4 \$ signs)
- rating: Minimum star rating (1-5)
- delivery: Whether restaurant delivers

Getting participants from previous screen:

```

const route = useRoute();
const { participants } = route.params;

```

This receives the list of selected people from WhosGoingScreen.

The filtering logic (handleNext):

```

const filteredRestaurants = parsedRestaurants.filter((restaurant) => {
  const matchesCuisine = !cuisine || restaurant.cuisine === cuisine;
  const matchesPrice = !price || Number(restaurant.price) <= Number(price);
  const matchesRating = !rating || Number(restaurant.rating) >= Number(rating);
  const matchesDelivery = !delivery || restaurant.delivery === (delivery === "Yes");

  return matchesCuisine && matchesPrice && matchesRating && matchesDelivery;
});

```

This is like a restaurant checklist:

Filter Rule	What It Means
!cuisine	If user picked "Any", ignore this filter
restaurant.cuisine === cuisine	If user picked a cuisine, restaurant must match
!price	If user picked "Any" price, ignore
Number(restaurant.price) <= Number(price)	Restaurant price must be LESS THAN OR EQUAL to max
!rating	If "Any" rating, ignore
Number(restaurant.rating) >= Number(rating)	Restaurant rating must be GREATER THAN OR EQUAL to min
!delivery	If "Any" delivery, ignore
restaurant.delivery === (delivery === "Yes")	Restaurant must match delivery preference

The AND (&&) means: ALL conditions must be true for a restaurant to pass.

If no restaurants match:

- Shows alert: "No restaurants match the selected criteria"
- User can go back and adjust filters

If restaurants match:

- Navigates to ChoiceScreen with both the participants list AND filtered restaurants

choiceScreen.js - The Main Event (Random Pick + Veto)

```

1 import React, { useState } from "react";
2 import {
3   View,
4   Text,
5   Modal,
6   ScrollView,
7   Alert,
8   TouchableOpacity,
9   FlatList,
10 } from "react-native";
11 import CustomButton from "../components/customButton";
12 import { useNavigation, useRoute } from "@react-navigation/native";
13
14

```

```

15 const getRandom = (min, max) => {
16   min = Math.ceil(min);
17   max = Math.floor(max);
18   return Math.floor(Math.random() * (max - min + 1)) + min;
19 };
20
21 const ChoiceScreen = () => {
22   const navigation = useNavigation();
23   const route = useRoute();
24   const { participantsList, filteredRestaurants } = route.params;
25
26   const [participants, setParticipants] = useState(participantsList);
27   const [restaurants, setRestaurants] = useState(filteredRestaurants);
28   const [chosenRestaurant, setChosenRestaurant] = useState(null);
29   const [selectedVisible, setSelectedVisible] = useState(false);
30   const [vetoVisible, setVetoVisible] = useState(false);
31   const [vetoDisabled, setVetoDisabled] = useState(false);
32   const [vetoText, setVetoText] = useState("Veto");
33
34   const selectRandomRestaurant = () => {
35     if (restaurants.length === 0) {
36       Alert.alert(
37         "No Restaurants Available",
38         "No restaurants match the selected criteria.",
39         [{ text: "Ok" } ]
40       );
41       return;
42     }
43
44     const index = getRandom(0, restaurants.length - 1);
45     const selected = restaurants[index];
46     setChosenRestaurant(selected);
47     setSelectedVisible(true);
48   };
49
50   const handleAccept = () => {
51     setSelectedVisible(false);
52     setVetoVisible(false);
53     navigation.navigate("PostChoiceScreen", { chosenRestaurant });
54   };
55
56

```

```

57 const handleVetoPress = () => {
58   setSelectedVisible(false);
59   setVetoVisible(true);
60 };
61
62

```

```

63 const handleVetoBy = (person) => {
64   const updatedParticipants = participants.map((p) =>
65     p.key === person.key ? { ...p, vetoed: "yes" } : p
66   );
67   const updatedRestaurants = restaurants.filter(
68     (r) => !r.key || r.key !== chosenRestaurant.key
69   );
70
71   if (updatedRestaurants.length === 0) {
72     Alert.alert("No options left", "All restaurants were vetoed.");
73     setVetoVisible(false);
74     return;
75   }
76
77   const stillCanVeto = updatedParticipants.some((p) => p.vetoed === "no");
78
79   setParticipants(updatedParticipants);
80   setRestaurants(updatedRestaurants);
81   setChosenRestaurant(null);
82   setSelectedVisible(false);
83   setVetoVisible(false);
84   setVetoDisabled(!stillCanVeto);
85   setVetoText(stillCanVeto ? "Veto" : "No Vetoes Left");
86
87   if (updatedRestaurants.length === 1) {
88     navigation.navigate("PostChoiceScreen", {
89       chosenRestaurant: updatedRestaurants[0],
90     });
91     return;
92   }
93 };
94
95

```

```

return (
  <View style={styles.container}>
    <FlatList
      style={styles.choiceScreenListContainer}
      data={participants.filter((x) => !!x)}
      keyExtractor={(item) => item?.key || Math.random().toString()}
      renderItem={({ item }) => (
        <View style={styles.choiceScreenListItem}>

```

```

        <Text style={styles.choiceScreenListItemName}>
            {item.firstName} {item.lastName} ({item.relationship})
        </Text>
        <Text>Vetoed: {item.vetoed || "no"}</Text>
    </View>
    )}
/>

<CustomButton
    text="Randomly Choose"
    width="94%"
    onPress={selectRandomRestaurant}
/>

<Modal
    visible={selectedVisible}
    animationType="slide"
    transparent={false}
>
    {chosenRestaurant ? (
        <View style={styles.selectedContainer}>
            <View style={styles.selectedInnerContainer}>
                <Text style={styles.selectedName}>{chosenRestaurant?.name}</Text>
                <View style={styles.selectedDetails}>
                    <Text style={styles.selectedDetailsLine}>
                        This is a {"★".repeat(chosenRestaurant?.rating || 0)} star
                    </Text>
                    <Text style={styles.selectedDetailsLine}>
                        {chosenRestaurant?.cuisine} restaurant
                    </Text>
                    <Text style={styles.selectedDetailsLine}>
                        with a price rating of{" "}
                        {"$".repeat(chosenRestaurant?.price || 0)}
                    </Text>
                    <Text style={styles.selectedDetailsLine}>
                        that {chosenRestaurant?.delivery ? "DOES" : "DOES NOT"}{" "}
                        deliver
                    </Text>
                </View>
            </View>

            <CustomButton text="Accept" width="94%" onPress={handleAccept} />
            <CustomButton
                text={vetoText}
                width="94%"
                onPress={handleVetoPress}
                disabled={vetoDisabled}
            />
        )
    )
}

```

```

        </View>
    </View>
) : (
    <View style={styles.selectedContainer}>
        <Text>No restaurant selected.</Text>
    </View>
) }
</Modal>

<Modal
    visible={vetoVisible}
    animationType="slide"
    transparent={false}
    onRequestClose={() => {}}
>
    <View style={styles.vetoContainer}>
        <View style={styles.vetoContainerInner}>
            <Text style={styles.vetoHeadline}>Who is vetoing?</Text>
            <ScrollView style={styles.vetoScrollViewContainer}>
                {participants
                    .filter((p) => p.vetoed === "no")
                    .map((p) => (
                        <TouchableOpacity
                            key={p.key}
                            style={styles.vetoParticipantContainer}
                            onPress={() => handleVetoBy(p)}
                        >
                            <Text style={styles.vetoParticipantName}>
                                {p.firstName} {p.lastName}
                            </Text>
                        </TouchableOpacity>
                    ))}
            </ScrollView>
            <View style={styles.vetoButtonContainer}>
                <CustomButton
                    text="Never Mind"
                    width="94%"
                    onPress={() => {
                        setVetoVisible(false);
                        setSelectedVisible(true);
                    }}
                />
            </View>
        </View>
    </Modal>
</View>

```

```

);
};

const styles = {
  container: {
    flex: 1,
    paddingTop: 20,
    paddingBottom: 20,
    gap: 20,
    justifyContent: "center",
    alignItems: "center",
  },
  headline: {
    fontSize: 30,
  },
  choiceScreenListContainer: {
    width: "94%",
  },
  choiceScreenListItem: {
    flexDirection: "row",
    marginTop: 4,
    marginBottom: 4,
    borderBottomWidth: 2,
    borderColor: "#e0e0e0",
    alignItems: "center",
  },
  choiceScreenListItemName: {
    flex: 1,
  },
  selectedContainer: {
    flex: 1,
    justifyContent: "center",
  },
  selectedInnerContainer: {
    alignItems: "center",
  },
  selectedName: {
    fontSize: 32,
  },
  selectedDetails: {
    paddingTop: 80,
    paddingBottom: 80,
    alignItems: "center",
  },
  selectedDetailsLine: {
    fontSize: 18,
  },

```



```

vetoContainer: {
  flex: 1,
  justifyContent: "center",
},
vetoContainerInner: {
  justifyContent: "center",
  alignItems: "center",
  alignContent: "center",
},
vetoHeadline: {
  fontSize: 32,
  fontWeight: "bold",
},
vetoScrollViewContainer: {
  height: "50%",
},
vetoParticipantContainer: {
  paddingTop: 20,
  paddingBottom: 20,
},
vetoParticipantName: {
  fontSize: 24,
},
vetoButtonContainer: {
  width: "100%",
  alignItems: "center",
  paddingTop: 40,
},
};

export default ChoiceScreen;

```

This is the main game show - where the drama happens!

The setup:

- Shows list of participants and whether they've vetoed
- Big "Randomly Choose" button

The random picker:

```

const getRandom = (min, max) => {
  min = Math.ceil(min);
  max = Math.floor(max);
  re

```

- Picks a random number between min and max
- Used to select a random restaurant from the filtered list

When "Randomly Choose" is pressed:

- Checks if there are any restaurants left
- Picks a random index

- Gets that restaurant
- Opens a modal showing the choice

The choice modal has:

- Restaurant name
- Details (rating, cuisine, price, delivery)
- Accept button - everyone agrees
- Veto button - someone objects

When Veto is pressed:

- Opens another modal showing people who HAVEN'T vetoed yet
- User picks who is vetoing
- That person's vetoed status changes to "yes"
- The chosen restaurant is REMOVED from the list
- If no restaurants left → game over alert
- If one restaurant left → automatically go to final screen
- If multiple left → pick again

The veto logic:

```
const updatedParticipants = participants.map((p) =>
  p.key === person.key ? { ...p, vetoed: "yes" } : p
);
```

- Find the person who vetoed
- Change their vetoed from "no" to "yes"

```
const updatedRestaurants = restaurants.filter(
  (r) => !!r && r.key !== chosenRestaurant.key
);
```

- Remove the vetoed restaurant from the list (can't pick it again)

Tracking veto status:

```
const stillCanVeto = updatedParticipants.some((p) => p.vetoed === "no");
.some() checks if ANY participant still has vetoed === "no"
```

- If true, someone can still veto
- If false, everyone has used their veto

postChoiceScreen.js - The Final Screen

```
1 import React from "react";
2 import { View, Text, StyleSheet, Button } from "react-native";
3 import { useRoute, useNavigation } from "@react-navigation/native";
4
5 const PostChoiceScreen = () => {
6   const route = useRoute();
7   const navigation = useNavigation();
8   const { chosenRestaurant } = route.params;
9
10  if (!chosenRestaurant) {
11    return (
12      <View style={styles.postChoiceScreenContainer}>
13        <Text style={styles.postChoiceHeadline}>No restaurant selected.</Text>
14      </View>
15    );
16  }
17 }
```

```

18   return () => {
19     <View style={styles.postChoiceScreenContainer}>
20       <View>
21         <Text style={styles.postChoiceHeadline}>Enjoy your meal!</Text>
22       </View>
23
24       <View style={styles.postChoiceDetailsContainer}>
25         <View style={styles.postChoiceDetailsRowContainer}>
26           <Text style={styles.postChoiceDetailsLabel}>Name:</Text>
27           <Text style={styles.postChoiceDetailsValue}>
28             {chosenRestaurant.name}
29           </Text>
30         </View>
31
32         <View style={styles.postChoiceDetailsRowContainer}>
33           <Text style={styles.postChoiceDetailsLabel}>Cuisine:</Text>
34           <Text style={styles.postChoiceDetailsValue}>
35             {chosenRestaurant.cuisine}
36           </Text>
37         </View>
38
39         <View style={styles.postChoiceDetailsRowContainer}>
40           <Text style={styles.postChoiceDetailsLabel}>Price:</Text>
41           <Text style={styles.postChoiceDetailsValue}>
42             {"$".repeat(chosenRestaurant.price)}
43           </Text>
44         </View>
45       </View>
46
47       <View style={styles.postChoiceDetailsRowContainer}>
48         <Text style={styles.postChoiceDetailsLabel}>Rating:</Text>
49         <Text style={styles.postChoiceDetailsValue}>
50           {"*".repeat(chosenRestaurant.rating)}
51         </Text>
52       </View>
53
54       <View style={styles.postChoiceDetailsRowContainer}>
55         <Text style={styles.postChoiceDetailsLabel}>Phone:</Text>
56         <Text style={styles.postChoiceDetailsValue}>
57           {chosenRestaurant.phone}
58         </Text>
59       </View>
60
61       <View style={styles.postChoiceDetailsRowContainer}>
62         <Text style={styles.postChoiceDetailsLabel}>Address:</Text>
63         <Text style={styles.postChoiceDetailsValue}>
64           {chosenRestaurant.address}
65         </Text>
66       </View>
67
68       <View style={styles.postChoiceDetailsRowContainer}>
69         <Text style={styles.postChoiceDetailsLabel}>Web Site:</Text>
70         <Text style={styles.postChoiceDetailsValue}>
71           {chosenRestaurant.website}
72         </Text>
73       </View>
74
75       <View style={styles.postChoiceDetailsRowContainer}>
76         <Text style={styles.postChoiceDetailsLabel}>Delivery:</Text>
77         <Text style={styles.postChoiceDetailsValue}>
78           {chosenRestaurant.delivery ? "Yes" : "No"}
79         </Text>
80       </View>
81     </View>
82
83     <View style={{ paddingTop: 80 }}>
84       <Button
85         title="All Done"
86         onPress={() => navigation.navigate("DecisionTimeScreen")}
87       />
88     </View>
89   );
90 };

```

```

92 const styles = StyleSheet.create({
93   postChoiceScreenContainer: {
94     flex: 1,
95     justifyContent: "center",
96     alignItems: "center",
97     alignContent: "center",
98   },
99   postChoiceHeadline: {
100     fontSize: 32,
101     paddingBottom: 80,
102   },
103   postChoiceDetailsContainer: {
104     borderWidth: 2,
105     borderColor: "black",
106     padding: 10,
107     width: "90%",
108   },
109   postChoiceDetailsRowContainer: {
110     flexDirection: "row",
111     justifyContent: "flex-start",
112     alignItems: "flex-start",
113     alignContent: "flex-start",
114     marginBottom: 12,
115   },
116   postChoiceDetailsLabel: {
117     width: 70,
118     fontWeight: "bold",
119     color: "red",
120   },
121   postChoiceDetailsValue: {
122     width: 300,
123   },
124 });
125
126 export default PostChoiceScreen;

```

This is the victory screen - you've finally chosen a restaurant!

The display:

- Big "Enjoy your meal!" headline
- A bordered box showing ALL restaurant details:
 - Name, cuisine, price, rating
 - Phone number, address, website
 - Delivery info

The "All Done" button:

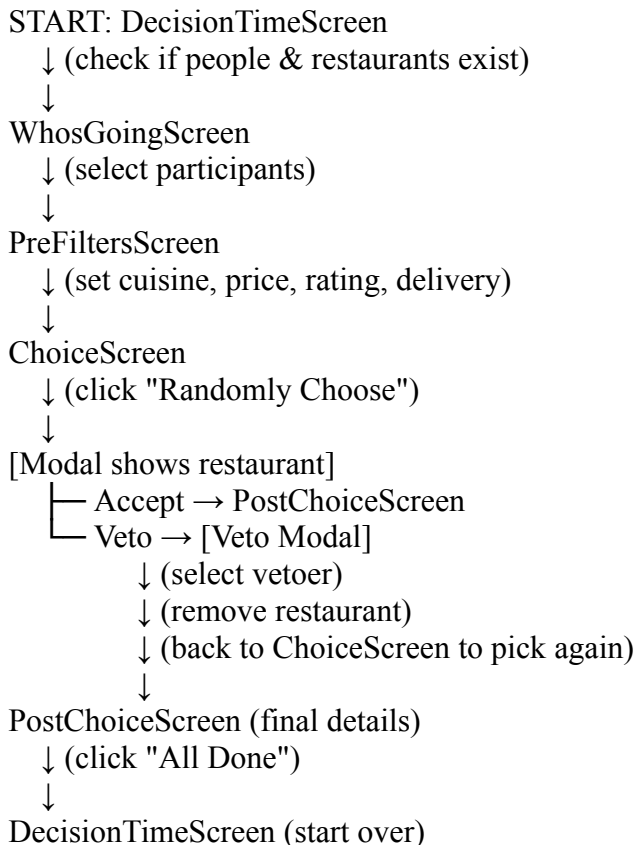
- Takes you back to the very beginning (DecisionTimeScreen)
- Ready to start the whole process again

The style trick:

```
<Text style={styles.postChoiceDetailsLabel}>Price:</Text>
<Text style={styles.postChoiceDetailsValue}>
  {"$".repeat(chosenRestaurant.price)}
</Text>
```

- {"\$".repeat(3)} creates "\$\$\$" - a visual way to show price level
- Same for rating with stars: {"★".repeat(4)} creates "★★★★"

The Complete Decision Flow



Evaluation

- 1. Make Sure Everything Runs**
 - Start your app (npx expo start or expo start)
 - Make sure it opens without any crashing
 - Check that all tabs (People, Decision, Restaurants) appear
- 2. Test the Restaurants Screen**
 - Add at least 2 new restaurants (like "McDonald's" and "Pizza Hut")
 - Fill in ALL the details for each:
 - Name, Cuisine, Price, Rating
 - Phone, Address, Website
 - Delivery option (Yes/No)
 - Go back to the list screen and VERIFY you see both restaurants

- Delete one restaurant (tap Delete, confirm "Yes")
- VERIFY it disappears from the list

3. Test the People Screen

- Add at least 3 new people (like "Mom", "Dad", "Friend")
- Fill in their details (first name, last name, relationship)
- Go back to the list screen and VERIFY you see all three
- Delete one person
- VERIFY they disappear from the list

4. Test the Decision Making Process (THE BIG ONE!)

Now for the fun part - actually using the app to choose a restaurant!

1. Go to the DECISION tab
2. Tap the food image to start
3. On "Who's Going?" screen:
 - Check ALL 3 people you added
 - Click "Next"
4. On "Pre-Filters" screen:
 - Try different filters (like "Italian" cuisine, "\$\$" price)
 - Click "Next"
5. On the main Choice screen:
 - Click "Randomly Choose"
 - When a restaurant appears, try the VETO button
 - Pick someone to veto
 - Click "Randomly Choose" again (should pick a different restaurant)
 - This time click ACCEPT
6. On the final screen:
 - VERIFY you see ALL restaurant details
 - Click "All Done" to return to start

5. Take Screenshots (Snapshots)

Take pictures of your app showing the decision process:

- Screenshot 1: "Who's Going?" screen with 3 people CHECKED
- Screenshot 2: "Pre-Filters" screen with some filters selected
- Screenshot 3: Choice screen showing a randomly selected restaurant
- Screenshot 4: Veto popup showing who can veto
- Screenshot 5: Final "PostChoiceScreen" with restaurant details

6. Save Your Screenshots

- Create a folder called "snapshots" in your project (if it doesn't exist)
- Inside "snapshots", create a folder called "week7-8"
- Put ALL your screenshots in that folder
- Name them something clear like:

- whos-going.jpg
- pre-filters.jpg
- choice-screen.jpg
- veto-popup.jpg
- final-details.jpg

7. Git Commands (Save Your Work)

Open your terminal and run these commands:

See what files you've changed

```
git status
```

Add ALL your changes

```
git add .
```

Commit with a message

```
git commit -m "Completed Week 7-8: Added decision screens and tested full flow"
```

Push to GitHub

```
git push origin main
```

8. Submit Your Work

- Go to your GitHub repository in a web browser
- Copy the URL (looks like: <https://github.com/yourusername/RestaurantChooser>)
- Go to your class portal (eLearn)
- Find the assignment submission
- PASTE your GitHub link
- Click Submit

